StashMind Completion Kit (Appwrite Edition)

Version 1.0, October 20, 2026. Goal: take StashMind from 'Tier 0 stable' to fully functional, deployable and ready for real users, on Appwrite, before any EasyTutor merge. Tier 2 stays with Software Factory.

0. Read this before you run anything

What Opencode's report tells us (an honest read)

- **Credit where due:** it says plainly that the app is not production-ready, and it names the three real blockers (no live database, placeholder secrets, no live end-to-end test).
- **Red flag 1: the test count did not move.** The earlier audit counted 26 tests. After 18 layers that added Telegram, M-Pesa, ask-the-source, study mode, Swahili and data-saver, the report still says 26. New behaviour with no new tests is unverified behaviour.
- **Red flag 2: 'real auth' on an in-memory store.** Accounts that vanish on restart are not real auth. It also reports building its own JWT login (`JWT_ACCESS_SECRET`). With Appwrite you should use Appwrite Auth and delete that code, not maintain a home-made login.
- **Red flag 3: the '17/17 stability checks' were written by the same agent.** Self-graded. Run the independent script (below).
- **Red flag 4: its capability table lists Tier 2 items** (model router, local TTS, shared core package). You assigned those to Software Factory. Check whether they exist and whether they are real, then decide whether to keep or park them.
- **Red flag 5: the model IDs.** The code uses `gemini-3.8-*` and `gemini-4-argon`. I could not find those in Google's docs or trackers; what they list are IDs such as a 3.6 and 3.7 Flash, a 3.1 Flash TTS preview, and a Live native-audio model with a retirement date in December 2026. Gemini 2.5 models were scheduled to shut down on October 16, 2026. Do not trust any model ID until it is checked against Google's live model list. The prompt below forces that check at startup.

Step zero, 10 minutes, no AI involved. Save `stashmind-doctor.sh` (the file shared with this kit) into the repo root and run `bash stashmind-doctor.sh --fix --boot`. It checks git state, install and gates, secrets, model IDs against Google's live list (if `GEMINI_API_KEY` is set), Appwrite integration, each Tier 0 and Tier 1 feature, docs, and boots the server to confirm that unauthenticated requests are rejected and that production refuses to start without configuration. It never commits or pushes. Keep the output as your baseline. Run it again at the end of every milestone; if it disagrees with an agent, the script wins.

Why Appwrite changes the plan (facts, from Appwrite's docs)

- Data lives in **TablesDB**: databases, tables, rows. Access is controlled by **table-level and row-level permissions** (turn on Row Security, grant create at table level, give each row `Role.user(id)` for its creator). There is no Postgres Row Level Security, so every earlier claim about RLS has to be rewritten.
- A resource created through a server SDK without explicit permissions is inaccessible to everyone by default. A server SDK with an API key acts as an administrator and bypasses user permissions, so the API key must be used sparingly.
- To keep user permissions in force, the backend should act **on behalf of the user with their JWT**, which Appwrite's server SDKs support.
- Appwrite Sites can serve a static single-page app, but site requests time out at a maximum of 30 seconds, and Functions have their own timeout settings. TTS and ingestion jobs are longer than that, so the plan keeps your Express API as a long-running Node service on a container host, with Appwrite Cloud (or self-hosted Appwrite) supplying Auth, TablesDB, Storage and Realtime.
- No multi-row transaction is assumed anywhere in the plan. Operations are idempotent and use unique indexes for deduplication.
- For live tests, run a local Appwrite stack with Docker (follow Appwrite's self-hosting guide). That is how 'no end-to-end live test' gets fixed without touching production.

---

1. How to run it

Which agent. The layers need a shell, git, Docker and the network, so **Opencode (or Software Factory driving Opencode) is the better runner**. AI Studio can do the code-only layers, but it cannot run a local Appwrite stack or push layer commits reliably. The prompt works in either; it makes the agent state what it can execute and mark the rest 'blocked' instead of pretending.

Decisions to make before you paste it

1. **Appwrite hosting:** Appwrite Cloud (less operations work, pick the region nearest your users after checking the current region list) or self-hosted on a VPS (fits your self-hosting interest, but you own backups, upgrades, email and security). My recommendation: Cloud for staging and production, local Docker for development and tests.
2. **API hosting target:** Google Cloud Run, Fly.io, Railway, Render or a VPS with Docker. Fill `[HOSTING_TARGET]`.

3. **Run mode:** `INTERACTIVE` (stops after every layer) or `UNATTENDED` (runs on until a human gate or a failure). Fill `[MODE]`. Start with INTERACTIVE for layers C00 to C05, then switch.

Human gates (things no agent can do; the prompt stops or works around them)

- **G1, before C02:** Appwrite project (staging), API key with only the scopes needed, database ID, your web hostnames added as platforms, email (SMTP) for verification mail. Local Appwrite via Docker for tests.
- **G2, before C07:** Gemini API key (use the paid tier before real users upload private documents).
- **G3, C10:** Safaricom Daraja sandbox credentials now; a Paybill or Till number and go-live approval for production.
- **G4, C11:** Telegram bot token from BotFather.
- **G5, C15:** a native Swahili speaker to review text and voice.
- **G6, C06 and C16:** you listening to real audio and testing voice on real phones.
- **G7, C21:** hosting account, domain, production secrets.
- **G8, C22:** legal review of the drafts.

---

2. The executive prompt

Paste this as the first message to the agent. Replace the bracketed values.

```
MISSION
Complete StashMind so that it is fully functional, deployable and ready for
real users (university students, life-long learners, professionals), on
Appwrite instead of Supabase. Finish StashMind completely BEFORE any
EasyTutor merge. Do not start Tier 2 work: model router, local TTS, shared
core package, EasyTutor merge, Software Factory briefings, workspaces for
other products. Park those as sprint files at the end.

SETTINGS
MODE: [INTERACTIVE or UNATTENDED]
HOSTING_TARGET: [Cloud Run / Fly.io / Railway / Render / VPS-Docker]
APPWRITE: [Cloud region or self-hosted URL]
Branch: completion/appwrite (create from main; never commit to main)

FIRST, STATE YOUR ENVIRONMENT
Say whether you can run shell commands, git, Docker, and make network calls
here. Anything you cannot execute must be marked BLOCKED, not assumed.

TRUTH RULES (non-negotiable)
```

1. Earlier reports of completion, including your own, are unverified claims. Verify against code and executed output. The Constitution rule stands: never claim a protection the code does not actually provide.
2. Statuses: VERIFIED (executed against real Appwrite, local Docker or a cloud test project, and passed), IMPLEMENTED-UNTESTED, PARTIAL, STUB-OR-FAKE, MISSING, BROKEN, DONE-PENDING-GATE (code and fake-based tests done, needs a human credential for live proof), UNVERIFIABLE.
3. Forbidden: dummy API keys, silent fallbacks that hide a failure, simulated dispatch, hard-coded return values, mock data on a production path, an in-memory store outside tests, tests that mock the thing under test, catching an error and reporting success.
4. Production must fail closed: if Appwrite, Gemini or any required secret is missing, the server refuses to start with a clear message.
5. Every feature layer must add tests for the new behaviour. Report the test count before and after. A feature layer with no new tests is rejected and redone.
6. Never weaken or skip a test, disable a check, or loosen a permission to make a gate pass.
7. Docs must match code. Remove every claim about Postgres, Supabase or Row Level Security. Never write 'production-ready' or '100% complete' unless the Definition of Done is met with evidence.
8. No real secrets in the repo, logs or reports.

COMMIT PROTOCOL (non-negotiable; I review history layer by layer)
- One commit per layer, made only after that layer's acceptance gates pass. Layers are the C00 to C23 items below.
- Format: type(layer-CNN-slug): imperative subject, 72 characters or fewer. Example: feat(layer-C04-auth): replace custom login with Appwrite Auth
- Body sections: Why / What / Requirements (F-numbers and tier items) / Evidence (exact commands and results) / Tests (added N, total M) / Human gates still open. End with the trailer line: Layer: CNN
- Do not mix layers. If you touched files belonging to another layer, split them with git add -p. Dependency and lockfile changes belong to the layer that needs them.
- Each layer commit includes its own updates to audit/matrix.json and docs/status/ for the items it touched. No separate status-only commits.
- Run npm run verify before every commit. Never commit red. Never use --no-verify.
- Push after every layer: git push -u origin completion/appwrite. Never force-push. Never amend or rewrite pushed commits. If a push is rejected, fetch, rebase, re-run the gates.
- A defect found later in a pushed layer gets a new commit: fix(layer-CNN-slug): subject.
- After each commit print the hash and the output of git show --stat --oneline HEAD.
- Tags: v0.9.0 after C05, v0.95.0 after C14, v1.0.0-rc1 after C23. Open a

```
pull request into main at the end but do not merge it.

BACKEND DECISION (write it as an ADR in C02 that supersedes the Supabase
decision)
- Appwrite Auth for identity. TablesDB for data. Storage for files. Realtime
for status updates. Express stays the API and the worker host, run as a long-
running Node service.
- Table-level and row-level permissions with Row Security on. Every row is
created with Role.user(owner) permissions, or a team role for shared
workspaces. No 'any' role except explicitly public demo content.
- Two clients only: a user-scoped client using the caller's JWT for
everything a user triggers, and an admin client using the API key, restricted
to a short ADR-listed set of system tasks (job queue, webhook deliveries,
usage counters, API-key lookup, payment callbacks, kill switch). Admin key
scopes are the minimum needed.
- No multi-row transaction assumptions. Use idempotency keys and unique
indexes. Respect Appwrite column and row size limits: store large JSON
(scripts) and all audio in Storage and keep only pointers in rows.

LAYERS (one commit each, in order)

C00 BASELINE. Create the branch. Pull. Run install, typecheck, lint, tests,
build and bash stashmind-doctor.sh. Record commands and outputs in
docs/status/BASELINE.md. For every Tier 0 and Tier 1 feature list the code
files, the tests that cover it and a status. Refresh audit/matrix.json to
reality. List features with zero tests. Fix nothing else. Acceptance:
BASELINE.md and refreshed matrix exist.

C01 HYGIENE. One lockfile (package-lock.json; remove bun.lock; short ADR).
Package name stashmind. engines node >=20 and .nvmrc. Scripts: typecheck,
lint (real ESLint, not tsc), test, test:live (opt-in), build, start, verify
(typecheck, lint, test, build). Resolve the vite/esbuild peer conflict
properly, no --force and no --legacy-peer-deps. CI workflow running verify.
Complete .env.example (names and comments only). .gitignore. Acceptance:
fresh clone, npm ci, npm run verify all green.

C02 APPWRITE FOUNDATION. ADR superseding Supabase (reasons, consequences,
permission model, data-modelling limits). Add node-appwrite and the appwrite
web SDK; remove @supabase packages and SUPABASE_* variables; move supabase/
SQL to docs/archive/. Config module validated with zod, fail-fast in
production. appwrite.config.json declaring database, tables, columns,
indexes, permissions and buckets. Idempotent scripts/provision-appwrite.ts
(create if missing, update on drift, never delete data) and scripts/check-
appwrite-schema.ts. Tables: profiles, sources, lessons, cards, saved_cards,
practice_sessions, activity_log, streaks, usage_counters, api_keys, webhooks,
webhook_deliveries, audit_log, jobs, payments, subscriptions, telegram_links,
```

card_feedback, ask_threads, system_flags. Buckets: uploads (private, TTL cleanup), audio (private), exports (private). Acceptance: provisioning run twice against local Appwrite changes nothing the second time; drift check passes.

C03 REPOSITORY LAYER. A domain-level Repository interface with no Appwrite types leaking out. AppwriteRepository using the two clients described above. MemoryRepository for tests only: it must refuse to construct when NODE_ENV is production. One contract test suite runs against both. Pagination with cursors, unique indexes for dedupe (for example content hash), retry on conflict. Acceptance: contract tests pass on both; live tests pass against local Appwrite; production refuses memory storage.

C04 AUTH. Replace the custom JWT and password code with Appwrite Auth: email and password, email verification, password recovery, Google OAuth, sessions via the web SDK. Server middleware verifies the caller's Appwrite JWT or session and builds the user context. Age confirmation (13 or older, 16 where required) stored in the profile and enforced server-side. Delete bcrypt/argon/jsonwebtoken usage and JWT_ACCESS_SECRET. Integrator API keys: stored hashed, prefix shown, scopes, per-key rate limit, rotation, audit log. Remove the demo user entirely. /me endpoint. Account deletion removes rows, files and the Appwrite user. Acceptance: real signup and login against local Appwrite; every unauthenticated request returns 401; tests for forged and expired tokens.

C05 PERMISSIONS AND ISOLATION. docs/security/PERMISSIONS.md with a matrix per table and bucket (who can create, read, update, delete). Row Security on everywhere. Two-real-user live tests: user B can neither read, list, update nor delete user A's rows or files, through the API and through the Appwrite SDK with B's session. API-key scope tests. Acceptance: tests green and the document matches appwrite.config.json. Tag v0.9.0.

C06 STORAGE AND AUDIO. Uploads to the private bucket with size and type limits and TTL cleanup. Real audio pipeline: Gemini multi-speaker TTS in chunks, assembled with ffmpeg into Opus (data-saver 24 to 32 kbps) plus a downloadable WAV or MP3; duration and section timestamps measured with ffprobe, not invented. Authenticated streaming endpoint with Range support so seeking works (or Appwrite time-limited file access if the current docs support it). Resume of failed chunks. Audio removed when a lesson is deleted. Acceptance: a real five-minute lesson yields playable audio whose measured duration matches the stored duration within 5 percent; a chunk-failure resume test passes. Human gate G6: someone listens.

C07 AI ENGINE. Remove the dummy key and every test-environment bypass. An AIClient interface with GeminiClient and FakeAiClient (tests use the fake through dependency injection). One model registry in config. At startup call

Google's models.list and fail closed in production if any configured model is absent. Do not assume gemini-3.8-* or gemini-4-argon exist; choose IDs only from the live list. Remove every hard-coded model ID outside the config module. Schema-enforced JSON, one repair retry, stored prompt versions. Log tokens and cost per call to a cost ledger. Timeouts, retries with backoff, 429 handling. docs/evals/ with 10 fixtures, an npm run eval script and a committed report. Automated grounding check: every card's sourceRef must resolve to a real span in the source text. Acceptance: unit tests with the fake; live smoke test and eval report (or DONE-PENDING-GATE G2).

C08 JOBS AND REALTIME. A jobs table with leases and heartbeats, retries with backoff, idempotency keys, per-step status (ingest, script, cards, tts, assemble, ready), step timeouts leading to a failed state with a readable message, crash recovery on restart, graceful shutdown. Worker loop in the API process or npm run worker. Appwrite Realtime subscription for status updates in the client, with polling as fallback. Acceptance: killing the worker mid-job and restarting resumes it; no job stays in progress past its timeout (tested).

C09 INGESTION HARDENING. Real fixtures: text PDF, multi-page PDF, scanned PDF (clear 'no text found' error), password-protected PDF; page and size limits. URL fetch: resolve DNS and block private, loopback, link-local and metadata ranges, re-validate every redirect, cap bytes and time, allow only text content types, readability extraction. YouTube: real transcript retrieval with a documented fallback, clear error when captions are absent, duration limit. Acceptance: SSRF tests for localhost, 127.0.0.1, 169.254.169.254, IPv6 loopback, decimal and hex IP forms, DNS rebinding through a mocked resolver, and redirects to private addresses.

C10 PAYMENTS (M-PESA). A BillingProvider interface and a Daraja provider (sandbox first): STK push, callback endpoint with an unguessable path and secret, payload validation, confirmation through the transaction status query before granting access, idempotent on CheckoutRequestID, persisted payments and subscriptions. One server-side entitlement module. KES price table in config. Student tier: verified email on a school domain list plus verified email, expiring yearly. Grace period, receipts. Acceptance: tests for replayed, tampered and duplicate callbacks; no premium access without a verified payment. Human gate G3 for the sandbox run.

C11 TELEGRAM BOT. Webhook verified with the secret token header. Linking: the app issues a one-time code, the user sends it to the bot. Commands: /start /help /lessons /language /datasaver /unlink. Accept URL, PDF (size cap) and text in private chats only; reply with progress, then the audio and top cards. Per-user rate limits. /unlink and account deletion remove the link. English and Swahili replies. Acceptance: simulated-update tests; live test on a test bot (gate G4).

C12 PWA, OFFLINE, DATA SAVER. Manifest, icons, service worker for the app shell. Offline lesson downloads in IndexedDB (audio, cards, transcript). Data-saver toggle: low-bitrate audio, no background downloads, text first. Background sync for saved cards. Install prompt. Acceptance: Playwright offline test passes; Lighthouse mobile 90 or more on key pages.

C13 ASK THE SOURCE. Question answering over the lesson's own source, with retrieval (chunking plus a verified embedding model, or direct long context). Every answer carries citations that point to exact spans. If the source does not support an answer, say so. The source text is data, never instructions (prompt-injection resistance). Threads stored per user. Acceptance: tests for answerable, unanswerable, injected-instruction and cross-user cases; on the eval set at least 90 percent of answers have valid citations.

C14 CARD QUALITY AND STUDY MODE. Thumbs up or down, edit, flag; show the source excerpt on each card. Quiz (multiple choice and cloze) and flashcard modes generated from saved cards. Persisted spaced-repetition scheduler with unit tests. Anki-compatible CSV export validated by an import fixture. Streak freeze. Acceptance: scheduler interval tests and export tests pass. Tag v0.95.0.

C15 SWAHILI AND I18N. UI string catalogue for English and Swahili, lesson language option, Swahili prompts, TTS voice support checked against real output. If voice quality is poor, fall back to text with device TTS and say so in the UI. Acceptance: tests; language choice persists; native-speaker review is gate G5 and the audio stays UNVERIFIABLE until done.

C16 VOICE PRACTICE (F13). The server mints short-lived credentials for the Live API so the main key never reaches the browser. Streaming with barge-in. Fallback to browser speech recognition plus TTS, then to text. Transcripts saved, limits enforced, safety rules kept. Acceptance: tests with a fake Live client; live test is gate G6.

C17 SECURITY HARDENING. helmet and a content security policy, CORS allowlist, HSTS, request size limits, rate limiting kept in a shared store (not process memory), webhook signing with timestamp and replay window, secret-rotation notes, npm audit gate for production dependencies with no high or critical findings (or a documented exception), secret scanning in CI, prompt-injection tests, audit logging, data export and deletion (Kenya Data Protection Act and GDPR), consent log, takedown endpoint and admin flow. Acceptance: security suite green.

C18 OBSERVABILITY AND COST. Structured logs with request IDs. /health (liveness) and /ready (checks Appwrite and required config). Optional error tracking via env. Cost ledger with daily and monthly caps and a kill switch

stored in system_flags (not in memory) that drops the app to text-only. Admin-only endpoint for cost per lesson, per user and failures. Acceptance: kill-switch test; /ready fails when Appwrite is unreachable.

C19 ACCESSIBILITY AND UX. The current code has no aria attributes: add labels, roles and focus management, keyboard navigation, contrast, reduced motion, transcripts and captions, form labels, the lang attribute, 360 px layouts, loading, empty and error states, onboarding with the three presets. Automated axe checks in CI with zero critical or serious issues.

C20 LIVE AND END-TO-END TESTS. A documented local Appwrite stack for tests. npm run test:live runs the live suite against it. Playwright end-to-end: sign up, verify, upload a real PDF, lesson appears, audio plays, save cards, ask a question, practise, streak updates, export, second user cannot see the first user's data, account deletion. Acceptance: green run log committed in docs/status/.

C21 DEPLOYMENT. Multi-stage Dockerfile (non-root, ffmpeg, healthcheck) and .dockerignore. Config for [HOSTING_TARGET]. The built frontend is served by the API. Separate staging and production Appwrite projects and secrets. GitHub Actions: CI on pull requests, deploy to staging on merge, deploy to production on a tag with manual approval, provisioning step in the pipeline. scripts/smoke.sh run after every deploy. Rollback procedure. Backup and restore procedure for the chosen Appwrite hosting. Domain, TLS, Appwrite platform hostnames, SMTP. Acceptance: staging URL live and smoke test passing (gate G7).

C22 LAUNCH READINESS. Legal drafts updated to name the real processors (Appwrite, Google Gemini, Safaricom, Telegram, the host) and to state the data-use terms of the Gemini tier in use. Landing page, KES pricing page, FAQ, support email, uptime check, changelog. Runbooks: incident, cost spike, backup restore drill, abuse and takedown. Ten real seed lessons. Beta invite flow. Privacy-respecting funnel analytics (activation, first lesson time, day-7 return, cost per active user). Update the launch checklist. Gate G8 for legal review.

C23 FINAL AUDIT AND HANDOFF. Re-run all gates and the doctor script. Refresh audit/matrix.json with evidence. Write docs/status/COMPLETION_REPORT.md: per requirement VERIFIED, DONE-PENDING-GATE or UNVERIFIABLE, with the evidence. Update AGENTS.md. Write sprint files for Software Factory: model router (Gemini, Anthropic, Groq, Ollama), local TTS, shared learning-core package, EasyTutor merge, Software Factory audio briefings, workspaces for other products, plus every open human gate. Tag v1.0.0-rc1. Open the pull request into main without merging.

HUMAN GATES

```
G1 Appwrite project, scoped API key, database ID, platform hostnames, SMTP.
G2 Gemini key. G3 Daraja credentials. G4 Telegram bot token. G5 Swahili
reviewer. G6 listening and device tests. G7 hosting, domain, production
secrets. G8 legal review. If a gate is not ready, build against fakes, mark
the item DONE-PENDING-GATE, say exactly what is needed, and continue with
layers that do not depend on it.

FAILURE PROTOCOL
Maximum three repair attempts per failing gate. Then stop, write the
diagnosis in docs/status/BLOCKERS.md, commit nothing broken, and report.

MODE BEHAVIOUR
INTERACTIVE: stop after every layer and wait for the word continue.
UNATTENDED: proceed layer by layer; stop only for a gate that blocks the next
layer's code, a failure after three attempts, or completion.

LAYER REPORT (end every layer with this)
LAYER, COMMIT (hash and subject), GATES (commands and exit codes), TESTS
(added, total), MATRIX CHANGES (ids and new statuses), HUMAN GATES OPEN,
RISKS, NEXT.

DEFINITION OF DONE (release candidate)
1. A fresh clone passes npm ci and npm run verify.
2. npm run test:live passes against a real Appwrite instance.
3. Two-user isolation tests pass.
4. The end-to-end journey passes.
5. Production refuses to start without required configuration.
6. Model IDs verified against Google's live list at startup.
7. M-Pesa sandbox payment and Telegram flows verified, or listed as DONE-
PENDING-GATE with exact needs.
8. Lighthouse mobile 90 or more and zero critical or serious axe issues.
9. Staging deployed and the smoke test passes.
10. Zero open S0 or S1 items in audit/matrix.json; everything else VERIFIED
or DONE-PENDING-GATE.

START
Begin with C00 now. State your environment first.
```

---

3. After the run

1. **Run the doctor script again** (`bash stashmind-doctor.sh --boot`) on the branch. Compare it with the agent's final report.
2. **Check the history is layered:** `git log --oneline completion/appwrite` should show one `layer-CNN` commit per layer, each small enough to review, with no force-push.

3. **Do the human gates:** keys, Daraja sandbox, Telegram test bot, listening tests, a Swahili reviewer, hosting and legal review.
4. **Independent re-audit:** run the re-audit prompt from the Audit Kit (Part D) in Software Factory's directory, against the final branch.
5. **Merge the pull request** only when the Definition of Done is met. Then Software Factory takes the Tier 2 sprint files: model router, local TTS, shared core, EasyTutor merge.

Bring back to me: the doctor output, `docs/status/COMPLETION_REPORT.md`, `git log --oneline`, and any BLOCKERS.md. I will compare them and point out where the story and the evidence differ.